252603912

BSc Computer Science

CPP Project report

I began this project on the 17th Of May 2026, and I completed on the 28th Of May 2026. In this report, I am explaining why I developed the program the way I did, how I was able to use pointers, arrays and functions in my program and some of the challenges I faced during the development of my program.

In the program, the user menu does not run once and closes. If I input a wrong value for the user menu, it allows me to re input a choice for a specific function. That is a switch case structure at work. If I had not used it, the program would always run once and close.

Also, I grouped all students information into one single class but did not create separate lists or variables. This is called encapsulation. I did this because I wanted to keep the data organized throughout the program so that the data doesn't get mixed up. Also, the whole project depended on arrays, pointers and functions.

In this project, I used functions to assign grades and remarks to the processed score instead of making the user type it manually. I did this to eliminate human errors and also save time as well. Imagine a user accidentally assigns someone's grade to a different person or awards an F to someone who might have had a total score of 87.0, this wouldn't have met the conditions of the project. Also, the program is guaranteed to give us 100% accuracy.

The global array studentRecords[30] acts as the master database for the application, setting aside thirty slots in the computer's memory to hold up to thirty individual student objects. Because an array doesn't automatically know how many slots are actually filled with real data, I had to use the tracker variable named studentCount. It starts at zero and increases by one every time a student is added.

In this project, the program was separated and broken into other functions. If I had written the entire project inside the main() function, it would have turned into a giant, confusing wall of text that would be incredibly hard to read or debug. If a bug popped up in the grading logic, I would have to search through hundreds of lines of code to find it. By splitting the code into distinct functions, each block has one specific job. If the statistics print out wrong, I know the issue is strictly inside displayStatistics.

I used pointers like Student* activeStudent and reference symbols like &count and &studentRecords[i]. Normally, when a variable is passed into a function, the computer creates a temporary copy of it, meaning any changes made inside the function

disappear when the function ends. By using a reference parameter like int &count, the function works directly on the original studentCount variable back in main(). Similarly, creating a pointer like Student* activeStudent = &studentRecords[count]; means the program isn't copying the student's data; it is holding the exact memory address of that student's slot in the array. Using the arrow operator (->) allows the function to write data directly into that specific memory slot.

The challenges I encountered is how to use the pointers and how to draw a flowchart so that I can see how to structure my program, so with the help of some few friends ,I was able to gain some assistance in putting the pieces of my flowcharts into one piece to see the flow of my data. For the pointers, I watched some videos on YouTube to help me.